

OptimalControl.jl: A Julia package for modeling and solving optimal control problems with ODEs

Jean-Baptiste Caillau ¹, Olivier Cots ^{2¶}, Joseph Gergaud ², Pierre Martinon ³, and Sophia Sed⁴

¹ Université Côte d'Azur, CNRS, Inria, LJAD, France ² Université Toulouse, CNRS, ENSEEIHT-IRIT, France ³ CAGE team, Inria Paris, France ⁴ Inria Sophia Antipolis Méditerranée, France ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

OptimalControl.jl (Caillau, Cots, Gergaud, Martinon, & Sed, 2024) is a Julia (Bezanson et al., 2017) package for modeling and solving optimal control problems governed by ordinary differential equations (ODEs). As the core of the [control-toolbox ecosystem](#), it provides a unified framework that supports both direct and indirect solution methods with applications spanning aerospace, medical imaging, epidemiology, and quantum control.

The package features an expressive domain-specific language (DSL) built around the `@def` macro, enabling users to define control problems using notation that closely resembles standard mathematical formulations. Problems are solved through direct transcription, converting the continuous problem into a nonlinear program (NLP) using discretization schemes including Euler, trapezoidal, midpoint, and high-order Gauss-Legendre collocation. Alternatively, indirect shooting methods based on Pontryagin's Maximum Principle can be employed. The architecture relies on a modeler-solver separation that provides a modular and extensible foundation, enabling GPU execution (currently limited to NVIDIA hardware) with minimal user intervention alongside standard CPU solvers.

The ecosystem includes extensive [tutorial resources](#), a benchmark problem collection ([OptimalControlProblems.jl](#) (Caillau, Cots, Gergaud, & Martinon, 2024) with formulations in OptimalControl DSL and [JuMP](#) (Lubin et al., 2023)), and performance comparison tools ([CTBenchmarks.jl](#)). Integration with Julia's ecosystem enables access to state-of-the-art tools: NLP solvers [IPOPT](#) (Wächter & Biegler, 2006) and [MadNLP.jl](#) (Shin et al., 2021, 2024), automatic differentiation and NLP modeling through JuliaSmoothOptimizers' [ADNLPModels](#) (Migot et al., 2023), GPU acceleration via [ExaModels.jl](#) (Shin et al., 2023), numerical integration from SciML's [DifferentialEquations.jl](#) (Rackauckas & Nie, 2017), and visualization through [Plots.jl](#) (Christ et al., 2023).

Problem Class

The package currently targets single-phase problems governed by explicit ODEs, written in Bolza form,

$$g(t_0, x(t_0), t_f, x(t_f), v) + \int_{t_0}^{t_f} f^0(t, x(t), u(t), v) dt \longrightarrow \min,$$

subject to the dynamics $\dot{x}(t) = f(t, x(t), u(t), v)$, where $x(t) \in \mathbb{R}^n$ is the state, $u(t) \in \mathbb{R}^m$ the control, and $v \in \mathbb{R}^q$ an optimisation variable, that is a finite-dimensional parameter

37 optimised together with the trajectory. Mayer ($f^0 = 0$) and Lagrange ($g = 0$) costs are
 38 the corresponding special cases, and maximisation is also supported. Both autonomous and
 39 non-autonomous dynamics are handled, the dependences being detected by the parser. The
 40 time bounds t_0 and t_f may be fixed or be components of v , which covers free initial and / or
 41 final time, in particular minimum time problems. Admissible constraints are of five types —
 42 boundary, variable, control, state, and mixed (control and state), the last three being *path*
 43 constraints, imposed at all times — and may be equalities or one- or two-sided inequalities,
 44 either box constraints on state, control and variable (for which dedicated bounds are passed
 45 to the NLP solver) or nonlinear. The variable may appear in any of them. The current
 46 implementation is restricted to *single-phase* problems with *explicit* ODE dynamics: multiphase
 47 problems, differential-algebraic equations, and hybrid or nonsmooth systems are not supported.
 48 Optimization of PDE or stochastic systems is out of scope as well (see the design trade-offs
 49 below).

50 Statement of Need

51 OptimalControl.jl is, to our knowledge, the only Julia package that unifies both direct and
 52 indirect methods for optimal control within a single, coherent framework.

53 This unified approach addresses a gap in the current optimal control software landscape.
 54 Existing tools are fragmented across programming languages and paradigms. Legacy packages
 55 such as [HamPath](#) (Caillau et al., 2012), [NutoPy](#) (Caillau et al., 2022), and [COTCOT](#) (Bonnard
 56 et al., 2007) implement sophisticated indirect methods but rely on Fortran implementations with
 57 MATLAB or Python interfaces, leading to complex and less extensible workflows. Proprietary
 58 solvers like [GPOPS-II](#) (Patterson & Rao, 2014) limit transparency and reproducibility. Open-
 59 source tools such as [BOCOP](#) (Bonnans et al., 2017), [ACADO](#) (Houska et al., 2011), and
 60 [acados](#) (Verschueren et al., 2021) provide valuable direct method implementations but are not
 61 natively integrated into a modern, high-level scientific computing ecosystem. Python-based
 62 tools such as [Dymos](#) (Falck et al., 2021), [Pyomo.DAE](#) (Nicholson et al., 2018), [do-mpc](#) (Fiedler
 63 et al., 2023), and [GEKKO](#) (Beal et al., 2018) offer complementary capabilities but are similarly
 64 limited to direct transcription approaches within the Python ecosystem.

65 Within Julia, most existing packages target specific domains: [RobustAndOptimalControl.jl](#) for
 66 linear systems, [QuantumControl.jl](#) (Machnes et al., 2025) and [Piccolo.jl](#) for quantum optimal
 67 control, [DirectTrajectoryOptimization.jl](#) for trajectory problems, [LinearMPC.jl](#) (Arnström et
 68 al., 2022) and [ModelPredictiveControl.jl](#) (Gagnon et al., 2024) for model predictive control,
 69 and [InfiniteOpt.jl](#) (Pulsipher et al., 2022) for infinite-dimensional optimization. In contrast to
 70 [InfiniteOpt.jl](#), which is designed as an extension of [JuMP.jl](#), our package provides a modeler-
 71 agnostic approach, accepting general Julia code and leveraging various optimization solvers.
 72 Moreover, contributing to general-purpose NLP modeling frameworks like [JuMP](#) or [CasADi](#)
 73 (Andersson et al., 2019) would not address the specific needs of optimal control: these tools
 74 lack native support for shooting methods based on Pontryagin's Maximum Principle, which
 75 heavily rely on differential geometric primitives.

76 OptimalControl.jl fills this gap by providing a DSL that matches mathematical notation, multiple
 77 discretization schemes and shooting methods, with planned support for homotopy continuation
 78 methods. The modeler-solver separation enables complementary use with [InfiniteOpt.jl](#) (JuMP-
 79 based) through alternative NLP modeling backends (currently [ADNLPModels](#), [ExaModels](#))
 80 and solvers. GPU acceleration and an ecosystem with domain-specific applications, tutorials,
 81 and benchmarking tools complete the offering. Target users include researchers and engineers
 82 working in optimal control, control theorists developing new algorithms, and students learning
 83 optimal control through interactive tutorials.

State of the Field

OptimalControl.jl requires Julia version 1.10 or later and is registered in the Julia General registry, enabling straightforward installation via `Pkg.add("OptimalControl")`. We compare it below with existing software.

- **Legacy tools (COTCOT, HamPath, NutoPy):** These Fortran packages excel at indirect methods and homotopy continuation but require multi-language setup (Fortran plus MATLAB / Python). OptimalControl.jl provides both direct and indirect methods in pure Julia with straightforward installation via package manager.
- **Direct-method tools (BOCOP, ACADO, GPOPS-II, acados, nosnoc):** Strong direct-method implementations: [GPOPS-II](#) delivers mature methods with MATLAB and C++ implementations but is proprietary; [acados](#) targets real-time MPC on embedded systems; [nosnoc](#) ([Nurkanović & Diehl, 2022](#)) specializes in nonsmooth optimal control; [CasADi](#), used as symbolic backend by several of these tools, is a general NLP modeler rather than an optimal control solver. OptimalControl.jl offers an open-source alternative with expressive DSL, native Julia ecosystem integration, GPU support, and unified direct and indirect approaches.
- **Julia packages:** [RobustAndOptimalControl.jl](#) targets linear systems; [QuantumControl.jl](#), [Piccolo.jl](#) and [DirectTrajectoryOptimization.jl](#) serve specific domains; [LinearMPC.jl](#) and [ModelPredictiveControl.jl](#) focus on model predictive control. [InfiniteOpt.jl](#) addresses a very rich range of problems, including optimization on PDEs or with chance constraints, focusing on direct transcription methods. OptimalControl.jl supports both CPU and GPU execution (currently NVIDIA-only, via [ExaModels.jl](#) + [MadNLP.jl](#)), and adds tools to do shooting in a unified framework plus systematic benchmarking through [OptimalControlProblems.jl](#) and [CTBenchmarks.jl](#).

Illustrative Example

A self-contained illustrative example is provided as a companion repository ([Caillau et al., 2026](#)), archived on Zenodo. It combines direct and indirect solution approaches for a constrained energy minimization problem: a direct method on a coarse grid identifies the three-arc structure (unconstrained–constrained–unconstrained) and initializes a shooting method based on Pontryagin's Maximum Principle, which then converges to arbitrary precision.

Software Design

The package architecture balances expressiveness, performance, and extensibility through modular design. The core is organized across internal packages within the [control-toolbox organization](#), each hosted as an independent Julia package and registered in the General registry: [CTBase.jl](#) defines the core types (optimal control models, solutions, initial guesses) and their accessors; [CTParser.jl](#) implements the `@def` DSL macro; [CTDirect.jl](#) handles direct transcription and NLP interfacing; and [CTFlows.jl](#) provides Hamiltonian flows and shooting for indirect methods. OptimalControl.jl re-exports these packages as a unified entry point. Contributors interested in a specific functionality can work directly on the relevant sub-package, each of which has its own documentation, tests, and continuous integration. The delegation of specific responsibilities is as follows:

- **DSL parsing:** [MLStyle.jl](#) ([Thautwarm & contributors, 2023](#)) enables pattern matching on abstract syntax trees generated from `@def` macro invocations. This design choice separates problem specification from solution methods, and allows a syntax as close as possible to the mathematical description of the problem.

- 129 ■ **Problem models and solutions:** Structured types represent optimal control models,
130 solutions, and initial guesses. Each type provides textual visualization, accessor methods
131 for introspection, and visual plotting capabilities for solutions. Extensible abstractions
132 enable addition of new problem classes without modifying core code.
- 133 ■ **Discretization:** Direct transcription converts continuous problems into NLPs. Supporting
134 multiple discretization schemes (Euler, midpoint, trapezoidal, Gauss-Legendre)
135 required careful abstraction to share transcription logic while allowing scheme-specific
136 implementations.
- 137 ■ **Modelers and solvers:** We chose a clear modeler-solver separation: discretization produces
138 [ADNLPMODELS](#) (able to deal with arbitrary user-defined functions) or [ExaModels](#) instances
139 compatible with multiple NLP solvers ([IPOPT](#) via [NLPModelsIpopt.jl](#) ([Orban et al., 2023](#)),
140 [Knitro](#) ([Byrd et al., 2006](#)), [MadNLP](#), and [Uno](#) ([Vanaret & Leyffer, 2026](#))). These
141 NLP solvers rely on linear solvers such as [MUMPS.jl](#) ([Amestoy et al., 2001, 2019](#);
142 [Montoisson et al., 2024](#)) for CPU computations or [CUDSS.jl](#) ([NVIDIA Corporation, 2025](#))
143 for GPU acceleration with MadNLP. Automatic differentiation via [ForwardDiff.jl](#) ([Revels
144 et al., 2016](#)) and [DifferentiationInterface.jl](#) ([Dalle & Hill, 2026](#)) avoids manual derivative
145 coding.
- 146 ■ **Indirect methods:** Hamiltonian flows integrate with [DifferentialEquations.jl](#) to access
147 adaptive stepping, event handling, and multiple ODE solvers without reimplementing.
148 Shooting methods rely on [NonlinearSolve.jl](#) ([Pal et al., 2025](#)) or [MINPACK.jl](#) ([Moré et al.,
149 1980](#)) for root-finding. Future extensions will incorporate homotopy continuation methods
150 leveraging bifurcation analysis tools like [BifurcationKit.jl](#) ([Veltz, 2020](#)). Currently, indirect
151 methods are limited to Hamiltonian flows, where the feedback control law depends on
152 both state and costate; support for non-Hamiltonian flows (open-loop or state-feedback
153 control laws) is under active development.
- 154 ■ **Testing and quality assurance:** The multi-repository structure is matched by a layered
155 testing strategy. Each sub-package is tested independently, combining unit tests,
156 integration tests, and code-quality checks, while [OptimalControl.jl](#) adds strong end-
157 to-end integration tests that solve complete optimal control problems by both direct and
158 indirect methods. Continuous integration runs across Linux, macOS, and Windows, on
159 both CPU and GPU (through a self-hosted CUDA runner), using reusable GitHub Actions
160 workflows centralized in [CTActions](#) and shared across the ecosystem. Code coverage
161 is tracked on Codecov, downstream packages and applications are guarded against
162 regressions through dedicated breakage tests triggered on pull requests, and beta versions
163 are distributed during development via a local registry, [ct-registry](#). Regression testing on
164 complex optimal control instances is increasingly supported by [OptimalControlProblems.jl](#),
165 which serves as a curated problem bank.

166 Key design trade-offs:

- 167 1. **DSL vs. programmatic API:** We prioritized a mathematical DSL to reduce cognitive load
168 for domain experts, accepting increased parsing complexity handled internally. Currently
169 we do not address optimization of PDE or stochastic systems.
- 170 2. **GPU acceleration strategy:** The modeler-solver separation enables a straightforward
171 CPU-to-GPU transition. Users select the [ExaModels](#) modeler with [MadNLP](#) and the
172 [CUDSS](#) linear solver for GPU execution, or simply append the `:gpu` token to the `solve` call.
173 This modular approach minimizes maintenance burden while enabling GPU performance
174 without reimplementing transcription logic. GPU support is currently limited to NVIDIA
175 hardware and to the [ExaModels](#) + [MadNLP](#) modeler-solver combination.
- 176 3. **Method coverage:** Supporting both direct and indirect approaches increases code
177 complexity but serves distinct user needs: direct methods for constrained problems with
178 many variables, indirect methods for theoretical analysis and smaller problems requiring
179 high accuracy. Both approaches rely on iterative solvers and may or may not converge,

180 e.g., depending on the initial guess. In the case of optimization solvers, the full output
181 status of the solver is returned allowing *a posteriori* analysis.

182 Research Impact Statement

183 OptimalControl.jl has been applied in published research across multiple domains. Independent
184 external adoption demonstrates the package's broader impact: Ferde et al. (2025) used it for
185 drone racing trajectory optimization, Caio & Silva (2025) for epidemiological models using
186 indirect methods, Morsky (2025) for vaccination strategies under social norms, Opmeer (2025)
187 for optimal harvesting of age-structured populations, and Evangelakos et al. (2025) for fast
188 charging protocols for quantum batteries.

189 Research by close collaborators further demonstrates the package's versatility across aerospace
190 (Herasimenka et al. (2026)), epidemiology (Bliman et al. (2026)), quantum control
191 (Beschastnyi et al. (2025)), microbial growth control (Innerarity Imizcoz et al. (2025)), and
192 theoretical optimal control (Bouali et al. (2025); Lutz & Privat (2025); Bayen et al. (2026);
193 Bonnard et al. (2026)).

194 The [control-toolbox organization](#) hosts domain-specific application packages built on
195 OptimalControl.jl: medical imaging optimization ([MagneticResonanceImaging.jl](#)), gene
196 regulatory networks ([PWLdynamics.jl](#)), spacecraft orbital transfers ([Kepler.jl](#)), epidemiological
197 modeling ([SIRcontrol.jl](#)), and variational calculus ([CalculusOfVariations.jl](#)).

198 The package also serves educational purposes through [Tutorials.jl](#), which covers topics from
199 linear-quadratic regulators to Model Predictive Control. These resources are used in academic
200 courses and workshops; the package was notably presented at JuliaCon 2023 ([Caillau et al., 2023](#)).
201 [OptimalControlProblems.jl](#) provides standardized test problems formulated in
202 both OptimalControl DSL and [JuMP](#), enabling systematic performance comparisons through
203 [CTBenchmarks.jl](#).

204 Development involves international collaborations with CNES (French space agency), Thales
205 Alenia Space, Inria, and CNRS. The package development began in September 2022, with
206 the first public release in February 2023. As of March 2026, OptimalControl.jl has published
207 more than 40 releases with contributions from multiple developers across the control-toolbox
208 ecosystem, demonstrating sustained community engagement.

209 AI Usage Disclosure

210 The core software implementation, algorithms, and architectural design of OptimalControl.jl
211 were developed by the authors. Generative AI tools—specifically Claude Sonnet (version 4.6)
212 and Claude Opus (version 4.8) by Anthropic—have been used to assist with code refactoring
213 and generation of unit and integration tests, always under human review. This paper was
214 drafted by the authors and subsequently revised with AI assistance for restructuring according
215 to JOSS format requirements and language editing. All technical content, examples, and
216 claims reflect the authors' work and judgment.

217 Acknowledgements

218 We thank the [control-toolbox community](#) and the broader Julia ecosystem for contributions
219 that shaped the package design and educational resources. Development has been supported
220 by partnerships with CNES, Thales Alenia Space, Inria, and CNRS. The software is distributed
221 under the MIT license (see [LICENSE](#)). We welcome community contributions following the
222 guidelines in [CONTRIBUTING.md](#).

References

- Amestoy, P. R., Buttari, A., L'Excellent, J.-Y., & Mary, T. (2019). Performance and Scalability of the Block Low-Rank Multifrontal Factorization on Multicore Architectures. *ACM Transactions on Mathematical Software*, 45(1), 2:1–2:26. <https://doi.org/10.1145/3242094>
- Amestoy, P. R., Duff, I. S., Koster, J., & L'Excellent, J.-Y. (2001). A fully asynchronous multifrontal solver using distributed dynamic scheduling. *SIAM Journal on Matrix Analysis and Applications*, 23(1), 15–41. <https://doi.org/10.1137/s0895479899358194>
- Andersson, J. A., Gillis, J., Horn, G., Rawlings, J. B., & Diehl, M. (2019). CasADi: A software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 11(1), 1–36. <https://doi.org/10.1007/s12532-018-0139-4>
- Arnström, D., Bemporad, A., & Axehill, D. (2022). A dual active-set solver for embedded quadratic programming using recursive LDL^T updates. *IEEE Transactions on Automatic Control*, 67(8), 4362–4369. <https://doi.org/10.1109/TAC.2022.3176430>
- Bayen, T., Bouali, A., & Bourdin, L. (2026). Minimum time problem for the double integrator with a loss-of-control region. *Nonlinear Analysis: Hybrid Systems*, 60, 101681. <https://doi.org/10.1016/j.nahs.2026.101681>
- Beal, L. D. R., Hill, D. C., Martin, R. A., & Hedengren, J. D. (2018). GEKKO optimization suite. *Processes*, 6(8), 106. <https://doi.org/10.3390/pr6080106>
- Beschastnyi, I., Dell'Elce, L., Pomet, J.-B., Sacchelli, L., & Tinoco, D. (2025). Pulse control of affine systems with applications to quantum control. *2025 IEEE 64th Conference on Decision and Control (CDC)*, 3769–3774. <https://doi.org/10.1109/CDC57313.2025.11312971>
- Bezanson, J., Edelman, A., Karpinski, S., & Shah, V. B. (2017). Julia: A fresh approach to numerical computing. *SIAM Review*, 59(1), 65–98. <https://doi.org/10.1137/141000671>
- Bliman, P.-A., Bouali, A., Loisel, P., Rapaport, A., & Virelizier, A. (2026). On the problem of minimizing the epidemic final size for SIR model by social distancing. *Mathematical Biosciences and Engineering*, 23(3), 567–593. <https://doi.org/10.3934/mbe.2026022>
- Bonnans, J. F., Martinon, P., Giorgi, D., Grélard, V., Maindrault, S., & Tissot, O. (2017). The bocop and BocHop optimization toolboxes. *IFAC-PapersOnLine*, 50(1), 8849–8854. <https://doi.org/10.1016/j.ifacol.2017.08.1745>
- Bonnard, B., Caillaud, J.-B., & Trélat, E. (2007). Second order optimality conditions in the smooth case and applications in optimal control. *ESAIM: Control, Optimisation and Calculus of Variations*, 13(2), 207–236. <https://doi.org/10.1051/cocv:2007012>
- Bonnard, B., Cots, O., & Privat, Y. (2026). The zermelo navigation problem on the 2-sphere of revolution: An optimal control perspective with applications to micromagnetism. *Nonlinearity*, 39(1), 015006. <https://doi.org/10.1088/1361-6544/ae2cd9>
- Bouali, A., Rapaport, A., & Bayen, T. (2025). Regularization of optimal control problems on stratified domains using additional controls. <https://hal.inrae.fr/hal-04928858>
- Byrd, R. H., Nocedal, J., & Waltz, R. A. (2006). Knitro: An integrated package for nonlinear optimization. In G. Di Pillo & M. Roma (Eds.), *Large-scale nonlinear optimization* (pp. 35–59). Springer US. https://doi.org/10.1007/0-387-30065-1_4
- Caillaud, J.-B., Cots, O., & Gergaud, J. (2012). Differential continuation for regular optimal control problems. *Optimization Methods and Software*, 27(2), 177–196. <https://doi.org/10.1080/10556788.2011.593625>
- Caillaud, J.-B., Cots, O., Gergaud, J., & Martinon, P. (2024). *OptimalControlProblems.jl: a collection of optimal control problems with ODEs in Julia*. <https://doi.org/10.5281/zenodo.17013180>

- 269 Caillau, J.-B., Cots, O., Gergaud, J., Martinon, P., & Sed, S. (2023). *On solving optimal*
270 *control problems with julia*. JuliaCon 2023, Cambridge, MA. Talk and abstract: [https://](https://pretalx.com/juliacon2023/talk/HDC8F7/)
271 pretalx.com/juliacon2023/talk/HDC8F7/. Video recording: [https://www.youtube.com/](https://www.youtube.com/watch?v=RYUtVnzLj5k)
272 [watch?v=RYUtVnzLj5k](https://www.youtube.com/watch?v=RYUtVnzLj5k).
- 273 Caillau, J.-B., Cots, O., Gergaud, J., Martinon, P., & Sed, S. (2024). *OptimalControl.jl: a*
274 *Julia package to model and solve optimal control problems with ODEs*. [https://doi.org/10.](https://doi.org/10.5281/zenodo.13336563)
275 [5281/zenodo.13336563](https://doi.org/10.5281/zenodo.13336563)
- 276 Caillau, J.-B., Cots, O., Gergaud, J., Martinon, P., & Sed, S. (2026). *Joss-oc-example:*
277 *Illustrative example companion to the OptimalControl.jl JOSS paper*. Zenodo. [https://](https://doi.org/10.5281/zenodo.19663649)
278 doi.org/10.5281/zenodo.19663649
- 279 Caillau, J.-B., Cots, O., & Martinon, P. (2022). Ct: Control toolbox – numerical tools and
280 examples in optimal control. *IFAC-PapersOnLine*, 55(16), 13–18. [https://doi.org/10.1016/](https://doi.org/10.1016/j.ifacol.2022.08.074)
281 [j.ifacol.2022.08.074](https://doi.org/10.1016/j.ifacol.2022.08.074)
- 282 Caio, P., & Silva, C. J. (2025). Application of indirect methods to optimal control problems in
283 epidemiology. *CONTROLO 2024*, 444–454. [https://doi.org/10.1007/978-3-031-81724-](https://doi.org/10.1007/978-3-031-81724-3_40)
284 [3_40](https://doi.org/10.1007/978-3-031-81724-3_40)
- 285 Christ, S., Schwabeneder, D., Rackauckas, C., Borregaard, M. K., & Breloff, T. (2023).
286 *Plots.jl – a user extendable plotting API for the julia programming language*. [https://](https://doi.org/10.5334/jors.431)
287 doi.org/10.5334/jors.431
- 288 Dalle, G., & Hill, A. (2026). A common interface for automatic differentiation. *Journal of*
289 *Machine Learning Research*, 27(25), 1–13. <http://jmlr.org/papers/v27/25-1024.html>
- 290 Evangelakos, V., Paspalakis, E., & Stefanatos, D. (2025). Fast protocols for charging a
291 three-spin-chain quantum battery. *Scientific Reports*, 15(1), 45626. [https://doi.org/10.](https://doi.org/10.1038/s41598-025-30354-1)
292 [1038/s41598-025-30354-1](https://doi.org/10.1038/s41598-025-30354-1)
- 293 Falck, R. D., Gray, J. S., Ponnappalli, K., & Wright, T. (2021). Dymos: A python package
294 for optimal control of multidisciplinary systems. *Journal of Open Source Software*, 6(59),
295 2809. <https://doi.org/10.21105/joss.02809>
- 296 Ferede, R., Blaha, T., Lucassen, E., De Wagter, C., & Croon, G. C. H. E. de. (2025). One net
297 to rule them all: Domain randomization in quadcopter racing across different platforms.
298 *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 6357–6363.
299 <https://doi.org/10.1109/ICRA55743.2025.11128790>
- 300 Fiedler, F., Karg, B., Lüken, L., Brandner, D., Heinlein, M., Brabender, F., & Lucia, S. (2023).
301 Do-mpc: Towards FAIR nonlinear and robust model predictive control. *Control Engineering*
302 *Practice*, 140, 105676. <https://doi.org/10.1016/j.conengprac.2023.105676>
- 303 Gagnon, F., Thivierge, A., Desbiens, A., & Bagge Carlson, F. (2024). *ModelPredictiveControl.jl:*
304 *advanced process control made easy in Julia*. <https://doi.org/10.48550/arXiv.2411.09764>
- 305 Herasimenka, A., Baresi, N., Green, L., Morgan, H., Underwood, C. I., Bridges, C., Jason,
306 S., Lucca Fabris, A., & Ryden, K. (2026). Low-thrust trajectory optimization for the
307 moon-enabled sun occultation mission concept. *AIAA SCITECH 2026 Forum*.
- 308 Houska, B., Ferreau, H. J., & Diehl, M. (2011). ACADO toolkit—an open-source framework for
309 automatic control and dynamic optimization. *Optimal Control Applications and Methods*,
310 32(3), 298–312. <https://doi.org/10.1002/oca.939>
- 311 Innerarity Imizcoz, J., Yabo, A. G., Djema, W., Francis, F., & Gouzé, J.-L. (2025). *Optimal*
312 *allocation control in microbial growth under a heat-shock*. [https://inria.hal.science/hal-](https://inria.hal.science/hal-05369609)
313 [05369609](https://inria.hal.science/hal-05369609)
- 314 Lubin, M., Dowson, O., Dias Garcia, J., Huchette, J., Legat, B., & Vielma, J. P. (2023).
315 JuMP 1.0: Recent improvements to a modeling language for mathematical optimization.

- 316 *Mathematical Programming Computation*. <https://doi.org/10.1007/s12532-023-00239-3>
- 317 Lutz, K., & Privat, Y. (2025). *Minimal time control of underdamped parametric oscillators*.
318 <https://hal.science/hal-05047678>
- 319 Machnes, S., Goerz, M. H., & contributors. (2025). *QuantumControl.jl: A framework for*
320 *quantum optimal control*. <https://juliaquantumcontrol.github.io/QuantumControl.jl>
- 321 Migot, T., Montoison, A., Orban, D., Soares Siqueira, A., & contributors. (2023).
322 *ADNLPMODELS.jl: Automatic Differentiation models implementing the NLPModels API*.
323 <https://doi.org/10.5281/zenodo.4605982>
- 324 Montoison, A., Orban, D., Sweeney, W. R., & contributors. (2024). *MUMPS.jl: A Julia*
325 *Interface to MUMPS* (Version 1.4.1). <https://doi.org/10.5281/zenodo.3271646>
- 326 Moré, J. J., Garbow, B. S., & Hillstom, K. E. (1980). *User guide for MINPACK-1* (ANL-80-74).
327 Argonne National Laboratory. <https://doi.org/10.2172/6997568>
- 328 Morsky, B. (2025). Vaccination and collective action under social norms. *Bulletin of*
329 *Mathematical Biology*, 87(5), 55. <https://doi.org/10.1007/s11538-025-01436-y>
- 330 Nicholson, B., Sirola, J. D., Watson, J.-P., Zavala, V. M., & Biegler, L. T. (2018). Pyomo.dae:
331 A modeling and automatic discretization framework for optimization with differential
332 and algebraic equations. *Mathematical Programming Computation*, 10(2), 187–223.
333 <https://doi.org/10.1007/s12532-017-0127-0>
- 334 Nurkanović, A., & Diehl, M. (2022). Nonsoc: A software package for numerical optimal
335 control of nonsmooth systems. *IEEE Control Systems Letters*, 6, 3110–3115. <https://doi.org/10.1109/lcsys.2022.3181800>
- 336 NVIDIA Corporation. (2025). *NVIDIA cuDSS (Preview): A high-performance CUDA Library*
337 *for Direct Sparse Solvers*. <https://docs.nvidia.com/cuda/cudss/index.html>.
- 338 Opmeer, M. (2025). Optimal harvesting of a continuously age-structured population with
339 density dependence. *PLOS ONE*, 20(9), 1–12. <https://doi.org/10.1371/journal.pone.0333087>
- 340 Orban, D., Soares Siqueira, A., & contributors. (2023). *NLPModelsIpopt.jl: A thin IPOPT*
341 *wrapper for NLPModels* (Version 0.10.1). <https://doi.org/10.5281/zenodo.2629034>
- 342 Pal, A., Holtorf, F., Larsson, A., Loman, T., Utkarsh, Schäfer, F., Qu, Q., Edelman, A., &
343 Rackauckas, C. (2025). NonlinearSolve.jl: High-performance and robust solvers for systems
344 of nonlinear equations in julia. *ACM Trans. Math. Softw.* <https://doi.org/10.1145/3779117>
- 345 Patterson, M. A., & Rao, A. V. (2014). GPOPS-II: A MATLAB software for solving multiple-
346 phase optimal control problems using hp-adaptive gaussian quadrature collocation methods
347 and sparse nonlinear programming. *ACM Transactions on Mathematical Software*, 41(1),
348 1–37. <https://doi.org/10.1145/2558904>
- 349 Pulsipher, J. L., Zhang, W., Hongisto, T. J., & Zavala, V. M. (2022). A unifying modeling
350 abstraction for infinite-dimensional optimization. *Computers & Chemical Engineering*, 156.
351 <https://doi.org/10.1016/j.compchemeng.2021.107567>
- 352 Rackauckas, C., & Nie, Q. (2017). Differentialequations.jl—a performant and feature-rich
353 ecosystem for solving differential equations in julia. *Journal of Open Research Software*,
354 5(1), 15. <https://doi.org/10.5281/zenodo.591257>
- 355 Revels, J., Lubin, M., & Papamarkou, T. (2016). *Forward-mode automatic differentiation in*
356 *Julia*. <https://arxiv.org/abs/1607.07892>
- 357 Shin, S., Anitescu, M., & Pacaud, F. (2024). Accelerating optimal power flow with GPUs: SIMD
358 abstraction of nonlinear programs and condensed-space interior-point methods. *Electric*
359

- 362 *Power Systems Research*, 236, 110651. <https://doi.org/10.1016/j.epsr.2024.110651>
- 363 Shin, S., Coffrin, C., Sundar, K., & Zavala, V. M. (2021). Graph-based modeling and
364 decomposition of energy infrastructures. *IFAC-PapersOnLine*, 54(3), 693–698. <https://doi.org/10.1016/j.ifacol.2021.08.322>
- 365
- 366 Shin, S., Pacaud, F., & Anitescu, M. (2023). *Accelerating optimal power flow with GPUs: SIMD abstraction of nonlinear programs and condensed-space interior-point methods*.
367 <https://doi.org/10.2139/ssrn.4601442>
- 368
- 369 Thautwarm, & contributors. (2023). *MLStyle.jl: Functional programming and pattern*
370 *matching for Julia*. <https://thautwarm.github.io/MLStyle.jl>
- 371 Vanaret, C., & Leyffer, S. (2026). *Implementing a unified solver for nonlinearly constrained*
372 *optimization*.
- 373 Veltz, R. (2020). *BifurcationKit.jl*. Inria Sophia-Antipolis. [https://hal.archives-ouvertes.fr/hal-](https://hal.archives-ouvertes.fr/hal-02902346)
374 [02902346](https://hal.archives-ouvertes.fr/hal-02902346)
- 375 Verschueren, R., Frison, G., Kouzoupis, D., Frey, J., Duijkeren, N. van, Zanelli, A., Novoselnik,
376 B., Albin, T., Quirynen, R., & Diehl, M. (2021). Acados – a modular open-source
377 framework for fast embedded optimal control. *Mathematical Programming Computation*.
378 <https://doi.org/10.1007/s12532-021-00208-8>
- 379 Wächter, A., & Biegler, L. T. (2006). On the implementation of an interior-point filter
380 line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*,
381 106(1), 25–57. <https://doi.org/10.1007/s10107-004-0559-y>